Struktury danych I
listy, stosy, kolejki, tablice 2D

Matura rozszerzona z informatyki – Sesja 7

Kacper Gębusia

Plan sesji

1. Listy
2. Stos (LIFO): nawiasy, notacja ONP
3. Kolejka (FIFO): zwykła i cykliczna
4. Tablice dwuwymiarowe
5. Którą strukturę wybrać?
6. Ćwiczenia

Powtórka z Sesji 6

1. Oblicz NWD i NWW liczb 84 i 36. Wykorzystaj algorytm Euklidesa:
   a) przez odejmowanie
   b) przez resztę z dzielenia

2. Prześledź wyszukiwanie binarne w liście $[4, 9, 14, 21, 26, 33, 40, 52, 61, 70, 85]$ dla wartości **61**. Wartość `mid` w każdym obrocie pętli wyznaczamy ze wzoru $\text{mid} = \left\lfloor \frac{\text{lo}+\text{hi}}{2} \right\rfloor$ (`(lo + hi) // 2`). Podaj kolejne wartości `mid` oraz wynik.

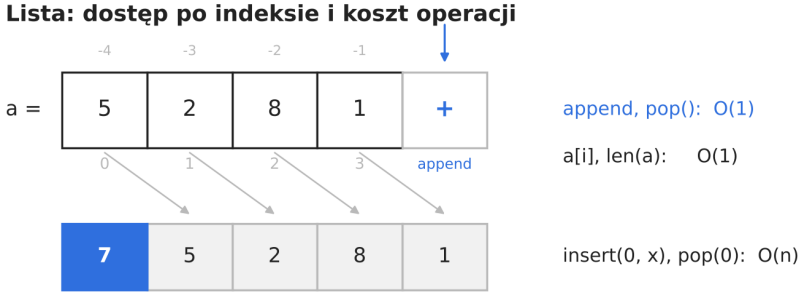
3. Ciąg Fibonacciego: $F(0) = 0$, $F(1) = 1$, $F(n) = F(n-1) + F(n-2)$.
   Ile wywołań wykona rekurencyjna funkcja obliczająca $F(6)$ wprost z definicji (**bez zapamiętywania wyników**), licząc wywołanie początkowe?

4. Wykonaj sito Eratostenesa dla $n = 30$: wypisz liczby pierwsze i podaj wartości i, dla których skreślamy wielokrotności.

Struktury danych, lista w Pythonie

Struktura danych – sposób przechowywania i organizowania danych wraz z operacjami na nich. Wybór struktury decyduje o złożoności algorytmu.

Lista w Pythonie to tablica dynamiczna: elementy leżą w pamięci obok siebie, dostęp po indeksie jest natychmiastowy, a lista sama się powiększa.



Operacje na liście i ich złożoność

Operacja	Przykład	Złożoność
dostęp po indeksie	`a[i]`	$O(1)$
długość	`len(a)`	$O(1)$
dopisanie na końcu	`a.append(x)`	$O(1)$
usunięcie z końca	`a.pop()`	$O(1)$
wstawienie w dowolne miejsce (np. na początek)	`a.insert(i, x)`	$O(n)$
usunięcie z początku / środka	`a.pop(0)`, `a.pop(i)`	$O(n)$
sprawdzenie, czy element jest w liście	`x in a`	$O(n)$
wycinek	`a[i:j]`	$O(j-i)$

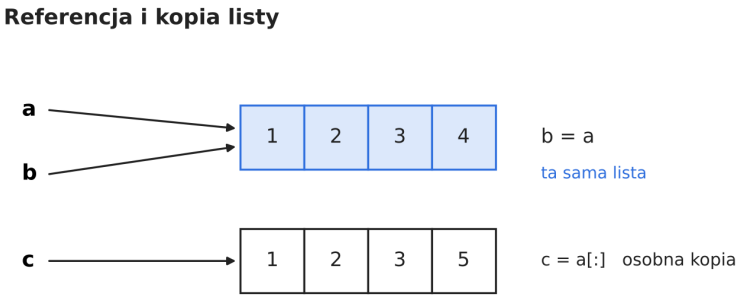
Operacje na początku listy są wolne, bo trzeba przesunąć wszystkie pozostałe elementy.

Operacje na liście – kod

```
a = [5, 2, 8, 1]
a.append(9)          # [5, 2, 8, 1, 9]
a.insert(0, 7)       # [7, 5, 2, 8, 1, 9]
x = a.pop()          # x = 9,  a = [7, 5, 2, 8, 1]
y = a.pop(0)         # y = 7,  a = [5, 2, 8, 1]
print(a[1:3])        # [2, 8]   (indeksy 1 i 2)
print(a[-1])         # 1        (ostatni element)
print(a[::-1])       # [1, 8, 2, 5]   (odwrocona kopia)
```

Referencja a kopia – idea

Przypisanie `b = a` **nie kopiuje** listy – obie nazwy wskazują tę samą listę. Kopię tworzy `a[:]` lub `a.copy()`.



Referencja a kopia – kod

```
a = [1, 2, 3]
b = a             # ta sama lista
c = a[:]          # kopia
b.append(4)
c.append(5)
print(a, b, c)    # [1, 2, 3, 4] [1, 2, 3, 4] [1, 2, 3, 5]
```

Przykład 1 – odwracanie listy w miejscu

Dwa wskaźniki, zamiana skrajnych elementów. Złożoność $O(n)$ (tyle zamian, co $\lfloor n/2 \rfloor$), pamięć $O(1)$.

```
def odwroc(a):
    i, j = 0, len(a) - 1
    while i < j:
        a[i], a[j] = a[j], a[i]
        i += 1
        j -= 1
    return a
```

`odwroc([1, 2, 3, 4, 5])` $\rightarrow$ `[5, 4, 3, 2, 1]`; `odwroc([6, 7, 8, 9])` $\rightarrow$ `[9, 8, 7, 6]`

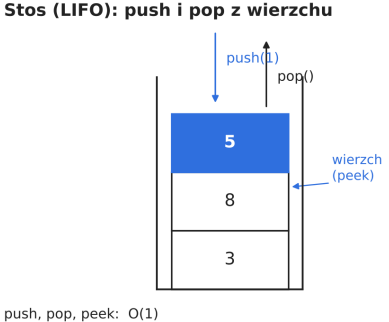
Przykład 2 – obrót cykliczny w lewo o k miejsc

```
def obrot(a, k):
    return a[k:] + a[:k]
```

`obrot([1, 2, 3, 4, 5, 6], 2)` $\rightarrow$ `[3, 4, 5, 6, 1, 2]`

Stos jako struktura abstrakcyjna – idea

Stos działa jak stos talerzy: elementy dokładamy i zdejmujemy **tylko z wierzchu**. Zasada **LIFO** (*last in, first out*) – ostatni włożony jest pierwszym wyjętym.



Operacja	Znaczenie
`push(x)`	połóż `x` na wierzchu
`pop()`	zdejmij i zwróć wierzch
`peek()`	podejrzyj wierzch
`pusty()`	czy stos jest pusty

Wszystkie operacje $O(1)$.

Realizacja stosu na liście – kod

Lista, na której wykonujemy wyłącznie operacje stosu: dokładanie na koniec (push) i zdejmowanie z końca (pop).

```
class Stos:
    def __init__(self):
        self.dane = []

    def push(self, x):
        self.dane.append(x)

    def pop(self):
        return self.dane.pop()

    def peek(self):
        return self.dane[-1]

    def pusty(self):
        return len(self.dane) == 0
```

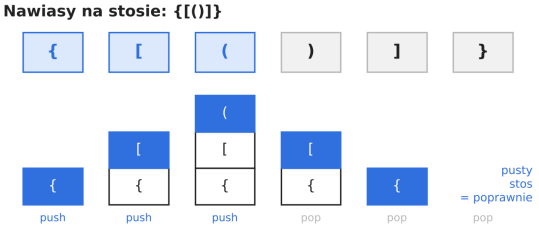
Stos w Pythonie

W Pythonie nie ma osobnego typu stosu – rolę stosu pełni zwykła lista (`append` = push, `pop()` = pop, wierzch to `stos[-1]`).

```
stos = []
stos.append(3)          # push(3)
stos.append(8)          # push(8)
print(stos[-1])         # peek -> 8
print(stos.pop())       # pop  -> 8
print(len(stos) == 0)   # pusty -> False
```

Przykład 3 – poprawność nawiasów: idea

W przykładach rolę stosu pełni zwykła lista. Otwierający nawias wkładamy na stos, a zamykający musi pasować do nawiasu z wierzchu. Na końcu stos musi być pusty. Złożoność $O(n)$.



Napis	Wynik	Dlaczego
`([]{})`	`True`	każdy nawias ma parę we właściwej kolejności
`([)]`	`False`	`)` trafia na `[`
`((`	`False`	na stosie zostają 2 nawiasy
`())(`	`False`	drugi `)` trafia na pusty stos

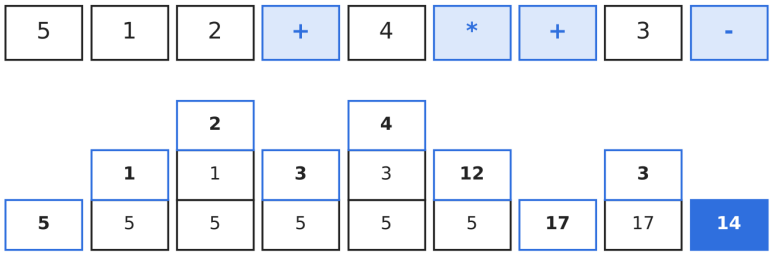
Poprawność nawiasów – kod

```
def nawiasy_ok(s):
    stos = []
    pary = {')': '(', ']': '[', '}': '{'}
    for c in s:
        if c in '([{':
            stos.append(c)
        elif c in pary:
            if not stos or stos.pop() != pary[c]:
                return False
    return len(stos) == 0
```

Przykład 4 – notacja ONP: idea

Operator zapisujemy **po** argumentach, więc nawiasy są zbędne: $(3 + 4) \cdot 2$ to $3\ \ 4\ \ +\ \ 2\ \ *$. Liczbę wkładamy na stos; operator zdejmuje dwa elementy (b, potem a), oblicza a op b i wkłada wynik.

ONP na stosie: 5 1 2 + 4 * + 3 -



operator zdejmuje b, potem a, wkłada a op b wynik: 14

Kolejność w odejmowaniu jest ważna: $a - b$, gdzie b zdjęto jako pierwsze.
Inne przykłady: $3\ \ 4\ \ +\ \ 2\ \ * = 14$; $7\ \ 2\ \ -\ \ 3\ \ * = 15$.

Obliczanie ONP – kod

```
def onp(wyrazenie):
    stos = []
    for t in wyrazenie.split():
        if t in ('+', '-', '*'):
            b = stos.pop()
            a = stos.pop()
            if t == '+':
                stos.append(a + b)
            elif t == '-':
                stos.append(a - b)
            else:
                stos.append(a * b)
        else:
            stos.append(int(t))
    return stos.pop()
```

Kolejka jako struktura abstrakcyjna – idea

Kolejka działa jak kolejka w sklepie: dokładamy na **końcu**, obsługujemy z **początku**. Zasada **FIFO** (*first in, first out*).



deque: append i popleft O(1); lista z pop(0) O(n)

Operacja	Znaczenie
`enqueue(x)`	dołóż `x` na końcu
`dequeue()`	zdejmij i zwróć element z początku
`pusta()`	czy kolejka jest pusta

Realizacje kolejki

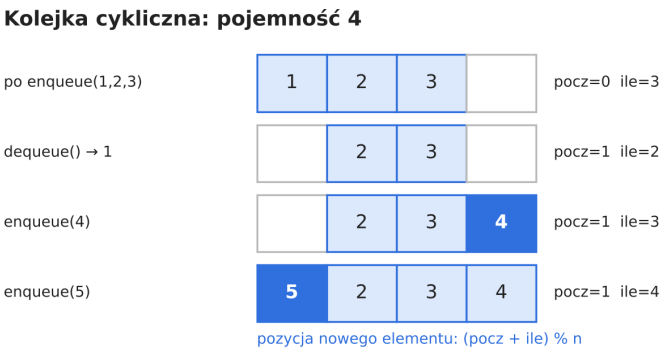
Kolejka na zwykłej liście (`append` + `pop(0)`) jest poprawna, ale `pop(0)` kosztuje $O(n)$.

Lepsze są dwie realizacje:

- kolejka cykliczna na tablicy,
- kolejka dwukierunkowa `deque`.

Kolejka cykliczna – idea

Tablica o stałym rozmiarze, indeks początku (`pocz`) i liczba elementów (`ile`). Nowy element trafia na pozycję `(pocz + ile) % n`, a po zdjęciu elementu `pocz` przesuwa się o 1 (modulo *n*). Tablica „zawija się”, a miejsce po zdjętych elementach jest używane ponownie.



Kolejka cykliczna – kod

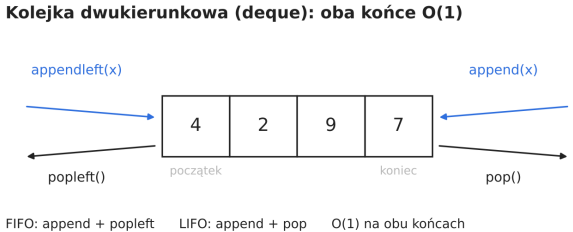
```
class KolejkaCykliczna:
    def __init__(self, n):
        self.t = [None] * n
        self.pocz = 0
        self.ile = 0

    def enqueue(self, x):
        if self.ile == len(self.t):
            raise OverflowError("kolejka pelna")
        self.t[(self.pocz + self.ile) % len(self.t)] = x
        self.ile += 1

    def dequeue(self):
        if self.ile == 0:
            raise IndexError("kolejka pusta")
        x = self.t[self.pocz]
        self.pocz = (self.pocz + 1) % len(self.t)
        self.ile -= 1
        return x
```

Kolejka dwukierunkowa (deque) – idea

Elementy dodajemy i zdejmujemy z **obu końców**. Kolejka FIFO i stos LIFO to jej szczególne przypadki.



Operacja	Znaczenie	Złożoność
`append(x)`	dodaj na końcu	$O(1)$
`appendleft(x)`	dodaj na początku	$O(1)$
`pop()`	zdejmij z końca	$O(1)$
`popleft()`	zdejmij z początku	$O(1)$

Kolejka w Pythonie – `collections.deque`

Jako zwykła kolejka (FIFO): tylko `append` (enqueue) i `popleft` (dequeue).

```
from collections import deque

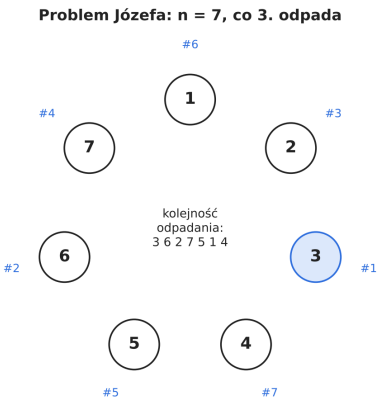
q = deque()
q.append(4)          # enqueue
q.append(9)
q.append(2)
print(q.popleft())   # dequeue -> 4
print(q)             # deque([9, 2])
```

Deque – operacje na drugim końcu

```
d = deque([2, 9])
d.appendleft(4)      # deque([4, 2, 9])
d.append(7)          # deque([4, 2, 9, 7])
print(d.pop())       # 7
print(d.popleft())   # 4
print(d)             # deque([2, 9])
```

Przykład 6 – problem Józefa: idea

n osób stoi w kole, co *k*-ta odpada. Kolejka: $k-1$ razy przenosimy osobę z początku na koniec, *k*-tą usuwamy. `josephus(7, 3)` $\to$ `[3, 6, 2, 7, 5, 1, 4]`; `josephus(5, 2)` $\to$ `[2, 4, 1, 5, 3]`.



Problem Józefa – kod

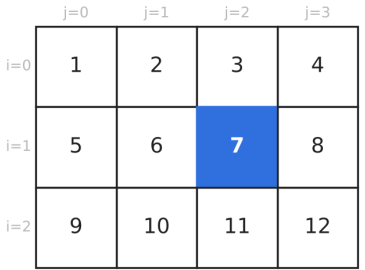
```
from collections import deque

def josephus(n, k):
    q = deque(range(1, n + 1))
    kolejnosc = []
    while q:
        for _ in range(k - 1):
            q.append(q.popleft())
        kolejnosc.append(q.popleft())
    return kolejnosc
```

Tablica dwuwymiarowa – idea

Tablica dwuwymiarowa (macierz) to lista list: `t[i][j]` to element z wiersza i i kolumny j (numeracja od 0). Macierz o n wierszach i m kolumnach ma $n \cdot m$ elementów, a przejście po wszystkich to $O(n \cdot m)$.

Tablica 2D: t[i][j] (wiersz i, kolumna j)



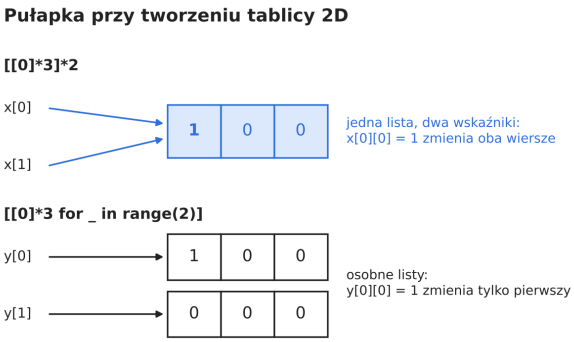
t[1][2] = 7
len(t) = 3 (wiersze)
len(t[0]) = 4 (kolumny)

Tablica 2D – tworzenie

```
t = [[1,  2,  3,  4],
     [5,  6,  7,  8],
     [9, 10, 11, 12]]          # 3 wiersze, 4 kolumny
print(len(t), len(t[0]))       # 3 4
print(t[1][2])                 # 7
```

Pułapka przy tworzeniu tablicy 2D

`[[0] * 3] * 2` tworzy dwa wiersze, które są **tą samą listą**.

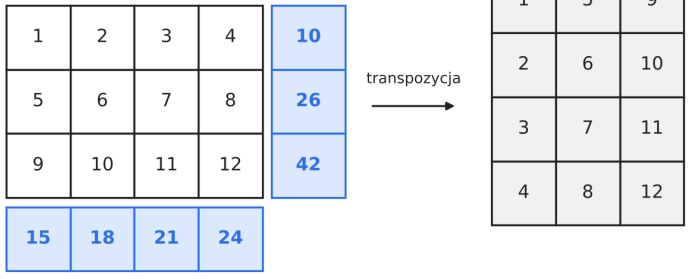


Pułapka – kod

```
x = [[0] * 3] * 2
x[0][0] = 1
print(x)        # [[1, 0, 0], [1, 0, 0]]   <- blad!

y = [[0] * 3 for _ in range(2)]
y[0][0] = 1
print(y)        # [[1, 0, 0], [0, 0, 0]]   <- poprawnie
```

Sumy wierszy i kolumn, transpozycja



sumy wierszy → prawa kolumna, sumy kolumn → dolny wiersz

t[i][j] → w[j][i] 3×4 → 4×3

Sumy i transpozycja – kod

```
n, m = len(t), len(t[0])

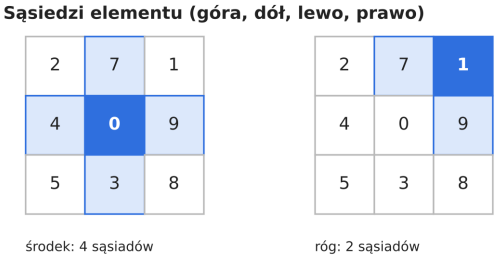
sumy_wierszy  = [sum(w) for w in t]                                  # [10, 26, 42]
sumy_kolumn   = [sum(t[i][j] for i in range(n)) for j in range(m)]   # [15, 18, 21, 24]
transpozycja  = [[t[i][j] for i in range(n)] for j in range(m)]
# [[1, 5, 9], [2, 6, 10], [3, 7, 11], [4, 8, 12]]  (macierz 4 x 3)
```

Macierz kwadratowa

- macierz $n \times n$: przekątna główna to `t[i][i]`, przekątna poboczna to `t[i][n-1-i]`
- dla macierzy 4×4 wypełnionej kolejno liczbami $1, \ldots, 16$ obie sumy wynoszą **34** ($1 + 6 + 11 + 16$ oraz $4 + 7 + 10 + 13$)
- macierz jest **symetryczna**, gdy `t[i][j] == t[j][i]` dla wszystkich i, j

Sąsiedzi elementu – idea

Często trzeba sprawdzić 4 sąsiadów (góra, dół, lewo, prawo) i **nie wyjść poza tablicę**. Dla $M = \begin{bmatrix} 2 & 7 & 1 \\ 4 & 0 & 9 \\ 5 & 3 & 8 \end{bmatrix}$: sąsiedzi `M[1][1]` to $[7, 3, 4, 9]$, a `M[0][2]` (róg) to $[9, 7]$.



Sąsiedzi elementu – kod

```
def sasiedzi(t, i, j):
    wynik = []
    for di, dj in ((-1, 0), (1, 0), (0, -1), (0, 1)):
        ni, nj = i + di, j + dj
        if 0 <= ni < len(t) and 0 <= nj < len(t[0]):
            wynik.append(t[ni][nj])
    return wynik
```

Przykład 7 – szczyty: idea

Element jest **szczytem**, jeśli jest większy od wszystkich sąsiadów.

Dla `[[1,3,2],[4,9,5],[6,2,8]]` szczytami są: $(1,1)$ – 9, $(2,0)$ – 6 oraz $(2,2)$ – 8 (trzy szczyty).

Wczytywanie macierzy z pliku (każdy wiersz pliku to liczby oddzielone spacjami) pokazuje kod na następnym slajdzie.

Szczyty i wczytywanie macierzy – kod

```
def szczyty(t):
    wynik = []
    for i in range(len(t)):
        for j in range(len(t[0])):
            if all(t[i][j] > x for x in sasiedzi(t, i, j)):
                wynik.append((i, j, t[i][j]))
    return wynik

with open("dane.txt") as f:
    t = [list(map(int, linia.split())) for linia in f]
```

Którą strukturę wybrać?

Struktura	Dodawanie	Zdejmowanie	Zastosowanie
lista	koniec $O(1)$, początek $O(n)$	koniec $O(1)$, początek $O(n)$	dane indeksowane, sortowanie, wyszukiwanie
stos (LIFO)	$O(1)$	$O(1)$	nawiasy, ONP, cofanie, rekurencja, DFS
kolejka (FIFO)	$O(1)$	$O(1)$ (`deque`, cykliczna)	obsługa w kolejności zgłoszeń, BFS
tablica 2D	–	–	plansze, obrazy, mapy, macierze

Ćwiczenia na zajęciach (1/3)

1. Jaki będzie stan listy `a` i wartości zwrócone przez `pop`? `a = [4, 7, 1]`, `a.append(9)`, `a.insert(1, 5)`, `x = a.pop()`, `y = a.pop(0)`.
2. Co wypisze program? `a = [1, 2, 3]`, `b = a`, `c = a[:]`, `b.append(4)`, `c.append(5)`, `print(a, b, c)`
3. Na początku stos jest pusty. Wykonaj: `push(3)`, `push(8)`, `pop()`, `push(5)`, `push(1)`, `pop()`, `pop()`. Co zwróciły kolejne `pop`? Jaki jest stan stosu na końcu?
4. Sprawdź poprawność nawiasów: a) `{[()()]}` b) `(()` c) `([)]` – podaj wynik i krótko uzasadnij.

Ćwiczenia na zajęciach (2/3)

5. Oblicz wartość wyrażeń zapisanych w ONP: a) `6 2 3 + * 4 -` b) `7 2 - 3 *`
6. Kolejka jest pusta. Wykonaj: `enqueue(4)`, `enqueue(9)`, `enqueue(2)`, `dequeue()`, `enqueue(7)`, `dequeue()`, `enqueue(1)`. Co zwróciły kolejne `dequeue`? Jaka jest zawartość kolejki (od początku)?
7. Kolejka cykliczna o pojemności 5. Wykonaj: `enqueue(1..4)` (cztery razy), dwa razy `dequeue()`, potem `enqueue(5)`, `enqueue(6)`, `enqueue(7)`. Podaj tablicę `t`, `pocz` i `ile`.

Ćwiczenia na zajęciach (3/3)

8. Dla macierzy $M = \begin{bmatrix} 2 & 7 & 1 \\ 4 & 0 & 9 \\ 5 & 3 & 8 \end{bmatrix}$ podaj: sumy wierszy, sumy kolumn, sumę przekątnej głównej i pobocznej oraz transpozycję.
9. Dla macierzy M z zadania 8 podaj sąsiadów (góra, dół, lewo, prawo) elementów `M[2][0]` oraz `M[1][1]`.
10. Co jest nie tak z `t = [[0] * 3] * 3`? Jak to poprawić? Jaka jest złożoność `x in a` dla listy, a jaka `dequeue` w `deque`?

Praca domowa (1/3)

1. Napisz funkcję `usun_duplikaty(a)` zwracającą listę bez powtórzeń z zachowaniem pierwotnej kolejności (użyj zbioru `set` do zapamiętywania widzianych elementów). Podaj wyniki dla `[3, 1, 3, 2, 1]` i `[5, 5, 5, 2, 5, 2]` oraz złożoność.
2. Napisz funkcję `odwroc_napis(s)` odwracającą napis z użyciem stosu. Co zwróci dla `"informatyka"`?
3. Oblicz wartości wyrażeń w ONP: a) `4 5 + 3 2 - *` b) `2 3 4 * + 6 -`. Zapisz w ONP wyrażenie $2 \cdot (3 + 4) - 5$ i oblicz je.

Praca domowa (2/3)

4. Sprawdź poprawność nawiasów dla napisów `(()())`, `())(` i `([]{})`. Dla niepoprawnych wskaż moment, w którym algorytm zwraca `False`.
5. Wykonaj `josephus(7, 3)` krok po kroku (stan kolejki po każdym usunięciu). W jakiej kolejności odpadają osoby? Kto zostaje ostatni?
6. Kolejka cykliczna o pojemności 4. Wykonaj: `enqueue(1..4)`, dwa razy `dequeue()`, `enqueue(5)`, `enqueue(6)`. Podaj `t`, `pocz`, `ile`.

Praca domowa (3/3)

7. Napisz funkcje `suma_wierszy(t)` i `transponuj(t)` dla macierzy $n \times m$. Podaj wyniki dla `[[1,2,3,4],[5,6,7,8],[9,10,11,12]]`.
8. Napisz funkcję `czy_symetryczna(t)` dla macierzy kwadratowej. Co zwróci dla `[[1,2,3],[2,5,6],[3,6,9]]` i dla `[[1,2,3],[2,5,6],[4,6,9]]`? Ile szczytów (elementów większych od wszystkich sąsiadów) ma macierz `[[1,3,2],[4,9,5],[6,2,8]]`?

Dziękuję za uwagę!